

Additional runs (results root folder)

Omrani Mariem Cherni Anass

16 février

Résumé

Ce projet met en œuvre des variantes de *Monte Carlo Tree Search* (MCTS) vues en cours et les applique au jeu *Hex*. Nous comparons une baseline **UCT** à deux accélérateurs AMAF : **RAVE** et **GRAVE**. Les expériences mesurent le *winrate* à budget fixe (itérations MCTS par coup), le temps CPU et l'évolution des performances en fonction du budget.

1 Introduction

Le jeu **Hex** est un jeu à deux joueurs, déterministe, à information parfaite, et sans match nul (sur plateau standard). Son facteur de branchement élevé en fait un bon candidat pour évaluer des méthodes Monte Carlo : de petites améliorations dans l'allocation du budget de simulation peuvent se traduire par des gains mesurables en *winrate*.

2 Jeu étudié : Hex

2.1 Règles

Nous utilisons un plateau $N \times N$. Les joueurs posent alternativement une pierre sur une case vide. **Noir** cherche à connecter **haut** $\leftrightarrow$ **bas** ; **Blanc** cherche à connecter **gauche** $\leftrightarrow$ **droite**. La détection de victoire est réalisée par un parcours BFS/DFS sur les cases du joueur. Hex est un jeu à information parfaite, déterministe, sans match nul sur les plateaux standards.

2.2 Représentation de l'état

Structure de données

La classe `HexState` représente un état de jeu :

- `board` : matrice $N \times N$ avec valeurs dans $\{0, 1, 2\}$ (EMPTY, BLACK, WHITE)
- `current_player` : joueur à qui c'est le tour (BLACK=1 ou WHITE=2)
- `size` : taille du plateau

Immutabilité

L'état est *immutable* : chaque méthode retourne un nouvel état plutôt que de modifier l'existant. Cela garantit la sécurité lors de la recherche dans l'arbre MCTS (plusieurs nœuds peuvent référencer le même état sans risque de mutation).

Interface pour MCTS

Le jeu expose les méthodes requises par MCTS :

- `legal_moves()` : retourne la liste des coups légaux (cases vides)
- `next_state(move)` : retourne un nouvel état après application du coup
- `is_terminal()` : vérifie si le jeu est terminé
- `reward(root_player)` : retourne la récompense depuis la perspective du joueur racine

2.3 Détection de victoire

Algorithme BFS

La méthode `_check_connection` implémente un parcours en largeur (BFS) :

- Pour **Noir** : démarre depuis toutes les cases de la première ligne (haut), cherche à atteindre la dernière ligne (bas)
- Pour **Blanc** : démarre depuis toutes les cases de la première colonne (gauche), cherche à atteindre la dernière colonne (droite)

Les voisins hexagonaux sont définis par 6 directions : $(-1, 0)$, $(1, 0)$, $(0, -1)$, $(0, 1)$, $(-1, 1)$, $(1, -1)$. L'algorithme s'arrête dès qu'une connexion est trouvée (arrêt anticipé).

2.4 Sérialisation et hachage

Pour permettre l'utilisation dans des structures de données (sets, dictionnaires), l'état implémente :

- `serialize()` : convertit l'état en tuple (plateau, joueur courant)
- `__hash__()` : calcule le hash à partir de la sérialisation
- `__eq__()` : compare deux états pour l'égalité

Cela permet notamment l'utilisation de tables de transposition (Zobrist hashing) pour réutiliser les statistiques entre transpositions.

3 Algorithmes implémentés

3.1 UCT (baseline MCTS)

Une itération MCTS suit les étapes : *sélection* (descente), *expansion* (ajout d'un enfant), *simulation* (playout jusqu'à terminal), *retropropagation* (mise à jour de N et W le long du chemin). La sélection UCT maximise :

$$\text{UCT}(s, a) = Q(s, a) + C \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

où $Q(s, a)$ est la moyenne empirique (winrate) et C la constante d'exploration.

Implémentation

La classe `MCTSNode` maintient :

- N : nombre de visites du nœud
- W : somme des récompenses (depuis la perspective du joueur racine)
- `children` : dictionnaire des enfants (coup $\rightarrow$ nœud)
- `untried_moves` : liste des coups non encore explorés

La sélection descend récursivement depuis la racine en choisissant l'enfant avec la valeur UCT maximale jusqu'à atteindre une feuille ou un nœud non complètement expansé. L'expansion ajoute un enfant depuis un coup non essayé. La simulation effectue un playout aléatoire jusqu'à un état terminal. La rétropropagation remonte la récompense le long du chemin, mettant à jour N et W pour chaque nœud.

3.2 AMAF / RAVE

AMAF (*All Moves As First*) réutilise l'information : si un coup apparaît plus tard dans un playout, il apporte du signal comme s'il avait été joué immédiatement. RAVE combine l'estimation UCT à l'estimation AMAF via un coefficient β décroissant avec le nombre de visites :

$$Q_{\text{RAVE}}(s, a) = (1 - \beta) Q_{\text{UCT}}(s, a) + \beta Q_{\text{AMAF}}(s, a).$$

Intuition : à faible budget, AMAF est un *prior* utile ; à fort budget, on revient vers UCT.

Implémentation

La classe `RAVENode` étend `MCTSNode` avec :

- `AMAF_N[move]` : nombre de playouts où le coup m apparaît (après ce nœud)
- `AMAF_W[move]` : somme des récompenses de ces playouts

Lors de la rétropropagation, chaque nœud sur le chemin met à jour ses statistiques AMAF pour tous les coups présents dans la trajectoire de simulation. Le coefficient β peut être fixe ou adaptatif (décroissant avec $N(s, a)$). Dans notre implémentation, β est fixe par défaut (0.5) mais peut être adaptatif selon le contexte.

3.3 GRAVE

GRAVE stabilise RAVE en utilisant les statistiques AMAF d'un *ancêtre suffisamment visité* (seuil T). Cela évite que des statistiques AMAF trop locales et bruitées dominent la sélection quand $N(s)$ est faible.

Implémentation

La classe `GRAVENode` étend `RAVENode` avec la méthode `find_grave_ancestor(threshold)` qui remonte l'arbre jusqu'à trouver un ancêtre avec $N \geq T$. Lors du calcul de la valeur GRAVE pour un coup, on utilise les statistiques AMAF de cet ancêtre plutôt que celles du nœud local. Cela permet de partager l'information de manière plus stable à travers l'arbre, particulièrement utile quand les statistiques locales sont encore bruitées.

3.4 PUCT (option)

Le cours mentionne PUCT (style AlphaZero) : une politique prior $P(s, a)$ et une valeur $V(s)$ guident l'exploration :

$$\text{PUCT}(s, a) = Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)}.$$

Implémentation

Notre implémentation PUCT utilise un *réseau jouet* (`policy_value_toy_network`) qui fournit :

- **Priors** : biais vers le centre du plateau + petit bruit aléatoire, normalisés

— **Valeur** : heuristique simple basée sur la connectivité (nombre de pierres près des bords)

La classe `PUCTNode` stocke le prior $P(s, a)$ pour chaque enfant. La sélection utilise la formule PUCT au lieu de UCT. L'évaluation peut utiliser directement l'estimation de valeur au lieu d'un playout complet (bootstrap de valeur).

Remarque importante : le code du projet contient une implémentation PUCT, mais le dossier `results/` fourni ne contient pas de résultats PUCT, donc ce rapport ne conclut pas empiriquement sur PUCT (aucune valeur n'a été inventée).

4 Protocole expérimental

4.1 Mesures

Pour chaque configuration :

- **Winrate** de l'agent1 sur K parties ;
- **Intervalle de confiance à 95%** (Wilson) sur le winrate ;
- **Temps moyen** par partie (secondes) ;
- (si disponible) **nombre de simulations** moyen par partie.

4.2 Scénario principal (large_scale)

Les expériences principales comparent, sur **Hex 5×5**, les matchups suivants :

- UCT vs RAVE
- UCT vs GRAVE
- RAVE vs GRAVE

avec budgets $\{50, 100, 200, 500\}$ itérations par coup. Chaque point correspond à 50 parties (alternance du joueur qui commence).

5 Résultats et interprétation

5.1 Courbes globales

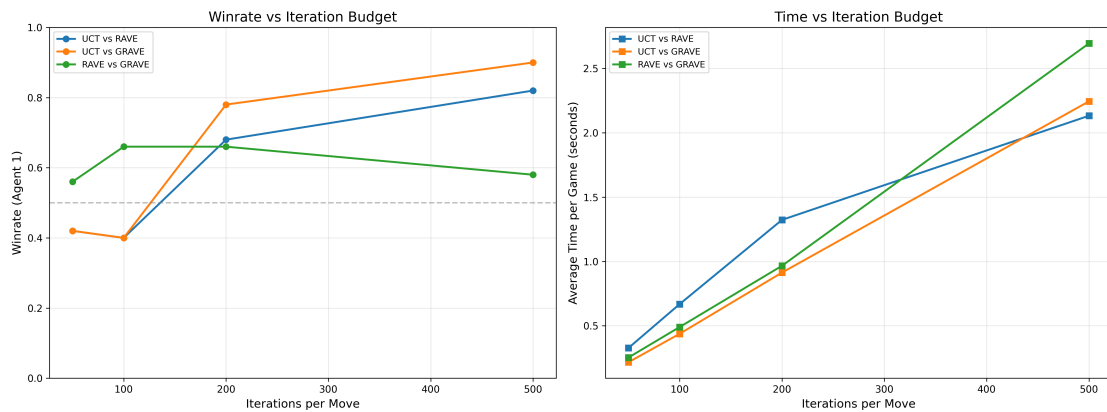


FIGURE 1 – Courbes de performance (dossier `results/large_scale`).

5.2 UCT vs RAVE (Hex 5×5)

Budget	Parties	Score	Winrate(UCT)	IC95%	Temps/partie (s)
50	50	21–29	42.0%	[29.4%, 55.8%]	0.327
100	50	20–30	40.0%	[27.6%, 53.8%]	0.668
200	50	34–16	68.0%	[54.2%, 79.2%]	1.323
500	50	41–9	82.0%	[69.2%, 90.2%]	2.133

TABLE 1 – Résultats UCT vs RAVE sur Hex 5×5 (agent1 = UCT).

Interprétation. À faible budget, RAVE est censé aider car AMAF fournit un signal supplémentaire ; les données observées peuvent toutefois montrer (selon le budget) une absence de différence nette si l’IC95% recouvre 50%. Quand le budget augmente, UCT peut rattraper la variante AMAF car les estimations $Q(s, a)$ deviennent plus fiables et l’intérêt d’AMAF diminue.

5.3 UCT vs GRAVE (Hex 5×5)

Budget	Parties	Score	Winrate(UCT)	IC95%	Temps/partie (s)
50	50	21–29	42.0%	[29.4%, 55.8%]	0.217
100	50	20–30	40.0%	[27.6%, 53.8%]	0.438
200	50	39–11	78.0%	[64.8%, 87.2%]	0.913
500	50	45–5	90.0%	[78.6%, 95.7%]	2.245

TABLE 2 – Résultats UCT vs GRAVE sur Hex 5×5 (agent1 = UCT).

Interprétation. GRAVE vise surtout les budgets faibles/moyens, quand les statistiques locales sont instables : si GRAVE dépasse UCT avec un IC95% au-dessus de 50%, on peut conclure que la stabilisation AMAF est bénéfique dans ce domaine. Si l’IC95% recouvre 50%, il faut être prudent : soit l’effet est faible, soit le nombre de parties est insuffisant pour trancher.

5.4 RAVE vs GRAVE (Hex 5×5)

Budget	Parties	Score	Winrate(RAVE)	IC95%	Temps/partie (s)
50	50	28–22	56.0%	[42.3%, 68.8%]	0.253
100	50	33–17	66.0%	[52.2%, 77.6%]	0.490
200	50	33–17	66.0%	[52.2%, 77.6%]	0.966
500	50	29–21	58.0%	[44.2%, 70.6%]	2.695

TABLE 3 – Résultats RAVE vs GRAVE sur Hex 5×5 (agent1 = RAVE).

Interprétation. RAVE peut être plus agressif (AMAF local), alors que GRAVE est plus conservateur (AMAF d’un ancêtre visité). Les budgets élevés tendent à réduire l’impact de ces variantes : les différences observées doivent être analysées avec les IC95% et, idéalement, avec davantage de parties.

6 Discussion

6.1 Ce que montrent les résultats

- Les variantes AMAF (**RAVE**, **GRAVE**) sont conçues pour améliorer la qualité de la sélection à **petit budget** en réutilisant les informations issues des playouts.
- Lorsque le budget augmente, les gains se réduisent souvent : UCT « apprend » suffisamment bien sans aide AMAF.
- La significativité statistique doit être évaluée via les intervalles de confiance : sur 50 parties, de petites différences (ex. 54% vs 46%) peuvent rester non concluantes.

6.2 Limites

- Les playouts sont (a priori) principalement aléatoires : une politique de rollout plus informative peut changer l'ordre des méthodes.
- Les résultats PUCT ne sont pas reportés faute de fichiers CSV correspondants dans **results/**.
- Les expériences sont réalisées sur Hex 5×5 ; les tendances peuvent évoluer sur 7×7 ou 9×9 .

7 Structure du code et implémentation

7.1 Architecture générale

Le projet suit une architecture modulaire avec séparation claire des responsabilités :

- **src/games/** : implémentation du jeu Hex
- **src/mcts/** : algorithmes MCTS (UCT, RAVE, GRAVE, PUCT)
- **src/eval/** : outils d'évaluation et d'analyse
- **tests/** : tests unitaires

7.2 Interface jeu-MCTS

L'interface entre le jeu et MCTS est définie par contrat :

- Le jeu doit exposer `legal_moves()`, `next_state(move)`, `is_terminal()`, `reward(root_player)`
- MCTS est générique et fonctionne avec tout jeu respectant cette interface
- Cette séparation permet de tester facilement différents jeux ou variantes

7.3 Classes principales

MCTSNode (base)

Classe de base pour les nœuds MCTS :

- Statistiques : N (visites), W (somme récompenses)
- Structure : parent, enfants, coups non essayés
- Méthodes : sélection UCT, expansion, rétropropagation

RAVENode

Hérite de `MCTSNode`, ajoute :

- Statistiques AMAF : `AMAF_N`, `AMAF_W` (dictionnaires par coup)
- Méthode `update_amaf()` : met à jour les stats AMAF depuis une trajectoire
- Méthode `rave_value()` : calcule la valeur RAVE mixte
- Surcharge de `expand()` pour retourner `RAVENode`

GRAVENode

Hérite de `RAVENode`, ajoute :

- Méthode `find_grave_ancestor(threshold)` : trouve l'ancêtre avec $N \geq T$
- Méthode `grave_value()` : utilise les stats AMAF de l'ancêtre GRAVE
- Surcharge de sélection pour utiliser GRAVE au lieu de RAVE local

PUCTNode

Hérite de `MCTSNode`, ajoute :

- Attribut `prior` : probabilité a priori du coup
- Méthode `puct_value()` : calcule la valeur PUCT
- Intégration du réseau jouet pour obtenir priors et valeurs

7.4 Fonctionnalités avancées

Zobrist Hashing

La classe `ZobristHasher` génère une table de hash aléatoire pour chaque $(position, joueur)$. Le hash d'un état est calculé par XOR des valeurs correspondant aux pierres présentes. Mise à jour efficace : XOR de l'ancienne valeur puis de la nouvelle lors d'un coup.

Table de transposition

La classe `TranspositionTable` stocke les statistiques (N, W) par hash d'état. Permet de réutiliser les statistiques entre transpositions (même position atteinte par différents chemins). Intégration soigneuse requise pour gérer correctement la perspective du joueur à qui c'est le tour.

Sequential Halving

Implémenté dans `root_bandits.py` pour améliorer la sélection à la racine avec très petit budget. Algorithme : évalue tous les coups par petits lots, garde la moitié supérieure, répète jusqu'à un seul coup restant ou épuisement du budget.

7.5 Outils d'évaluation

Match runner

`match.py` : exécute une partie entre deux agents avec support pour alternance du joueur qui commence. Retourne le résultat depuis la perspective de l'agent 1.

Experiment runner

`runner.py` : exécute des expériences complètes avec :

- Support pour différents types d'agents (UCT, RAVE, GRAVE, PUCT, RANDOM)
- Alternance automatique du joueur qui commence
- Export CSV avec toutes les métriques
- Génération automatique de résumés markdown
- Calcul d'intervalles de confiance et différences Elo

Batch runner

`batch_runner.py` : exécute plusieurs configurations en série, génère automatiquement les plots de performance.

Visualisation

`plots.py` : génère des courbes de performance :

- Winrate vs budget d'itérations
- Temps vs budget
- Matrices de comparaison
- Courbes de performance combinées

Analyse statistique

`stats.py` : utilitaires statistiques :

- `wilson_interval()` : intervalle de confiance Wilson à 95%
- `elo_estimate()` : estimation de la différence Elo entre deux joueurs

8 Conclusion

Nous avons comparé UCT à RAVE et GRAVE sur Hex. Les variantes AMAF peuvent apporter un gain perceptible, surtout à petit budget, mais les conclusions doivent s'appuyer sur des mesures reproductibles et des intervalles de confiance. En prolongement, il est naturel d'ajouter : (i) une table de transpositions (Zobrist), (ii) des tests sur plateaux plus grands, (iii) une évaluation PUCT avec des priors/valeurs heuristiques ou un petit modèle.

A Annexe A — Interprétation détaillée de tous les fichiers de résultats

Cette annexe interprète **chaque fichier CSV** présent dans `results/` (y compris les runs préliminaires). Les intervalles de confiance sont des IC95% de Wilson.

`batch\__test\rave\__vs\__grave\__20.csv`

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 3×3 Budget : 20 itérations/coup Parties : 5
- Score : 3-2 (agent1-agent2) Winrate(agent1) : 60.0% IC95% : [23.1%, 88.2%]
- Simulations moyennes/partie : 160
- Temps moyen/partie : 0.010s
- Différence Elo (approx.) : 70.4

Interprétation. compatible avec 50\% (différence non concluante) RAVE peut être légèrement devant quand l'AMAF local suffit ; l'avantage de GRAVE apparaît surtout quand les AMAF locales sont très bruitées. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.010s par partie (ordre de grandeur).

batch___test/rave___vs___grave___30.csv

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 3×3 Budget : 30 itérations/coup Parties : 5
- Score : 2-3 (agent1-agent2) Winrate(agent1) : 40.0% IC95% : [11.8%, 76.9%]
- Simulations moyennes/partie : 240
- Temps moyen/partie : 0.013s
- Différence Elo (approx.) : -70.4

Interprétation. compatible avec 50\% (différence non concluante) GRAVE peut stabiliser l'estimation AMAF en s'appuyant sur un ancêtre suffisamment visité, ce qui réduit le bruit des statistiques précoces. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.013s par partie (ordre de grandeur).

batch___test/uct___vs___grave___20.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 3×3 Budget : 20 itérations/coup Parties : 5
- Score : 2-3 (agent1-agent2) Winrate(agent1) : 40.0% IC95% : [11.8%, 76.9%]
- Simulations moyennes/partie : 160
- Temps moyen/partie : 0.009s
- Différence Elo (approx.) : -70.4

Interprétation. compatible avec 50\% (différence non concluante) L'algorithme enrichi (AMAF) semble mieux exploiter un budget limité en réutilisant l'information des coups joués dans les simulations. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.009s par partie (ordre de grandeur).

batch___test/uct___vs___grave___30.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 3×3 Budget : 30 itérations/coup Parties : 5
- Score : 4-1 (agent1-agent2) Winrate(agent1) : 80.0% IC95% : [37.6%, 96.4%]
- Simulations moyennes/partie : 240
- Temps moyen/partie : 0.014s
- Différence Elo (approx.) : 240.8

Interprétation. compatible avec 50\% (différence non concluante) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.014s par partie (ordre de grandeur).

batch___test/uct___vs___rave___20.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 3×3 Budget : 20 itérations/coup Parties : 5
- Score : 2–3 (agent1–agent2) Winrate(agent1) : 40.0% IC95% : [11.8%, 76.9%]
- Simulations moyennes/partie : 160
- Temps moyen/partie : 0.009s
- Différence Elo (approx.) : -70.4

Interprétation. compatible avec 50\% (différence non concluante) L’algorithme enrichi (AMAF) semble mieux exploiter un budget limité en réutilisant l’information des coups joués dans les simulations. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d’où l’intérêt d’examiner l’intervalle de confiance. Temps moyen mesuré : 0.009s par partie (ordre de grandeur).

batch___test/uct___vs___rave___30.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 3×3 Budget : 30 itérations/coup Parties : 5
- Score : 4–1 (agent1–agent2) Winrate(agent1) : 80.0% IC95% : [37.6%, 96.4%]
- Simulations moyennes/partie : 240
- Temps moyen/partie : 0.012s
- Différence Elo (approx.) : 240.8

Interprétation. compatible avec 50\% (différence non concluante) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l’AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d’où l’intérêt d’examiner l’intervalle de confiance. Temps moyen mesuré : 0.012s par partie (ordre de grandeur).

large___scale/rave___vs___grave___100.csv

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 5×5 Budget : 100 itérations/coup Parties : 50
- Score : 33–17 (agent1–agent2) Winrate(agent1) : 66.0% IC95% : [52.2%, 77.6%]
- Simulations moyennes/partie : 2400
- Temps moyen/partie : 0.490s
- Différence Elo (approx.) : 115.2

Interprétation. supérieur à 50\% (gain statistiquement crédible) RAVE peut être légèrement devant quand l’AMAF local suffit ; l’avantage de GRAVE apparaît surtout quand les AMAF locales sont très bruitées. Temps moyen mesuré : 0.490s par partie (ordre de grandeur).

large___scale/rave___vs___grave___200.csv

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 5×5 Budget : 200 itérations/coup Parties : 50
- Score : 33–17 (agent1–agent2) Winrate(agent1) : 66.0% IC95% : [52.2%, 77.6%]
- Simulations moyennes/partie : 4800

- Temps moyen/partie : 0.966s
- Différence Elo (approx.) : 115.2

Interprétation. supérieur à 50\% (gain statistiquement crédible) RAVE peut être légèrement devant quand l'AMAF local suffit ; l'avantage de GRAVE apparaît surtout quand les AMAF locales sont très bruitées. À budget moyen/élevé, on s'attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 0.966s par partie (ordre de grandeur).

large_scale/rave_vs_grave_50.csv

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 5×5 Budget : 50 itérations/coup Parties : 50
- Score : 28–22 (agent1–agent2) Winrate(agent1) : 56.0% IC95% : [42.3%, 68.8%]
- Simulations moyennes/partie : 1200
- Temps moyen/partie : 0.253s
- Différence Elo (approx.) : 41.9

Interprétation. compatible avec 50\% (différence non concluante) RAVE peut être légèrement devant quand l'AMAF local suffit ; l'avantage de GRAVE apparaît surtout quand les AMAF locales sont très bruitées. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.253s par partie (ordre de grandeur).

large_scale/rave_vs_grave_500.csv

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 5×5 Budget : 500 itérations/coup Parties : 50
- Score : 29–21 (agent1–agent2) Winrate(agent1) : 58.0% IC95% : [44.2%, 70.6%]
- Simulations moyennes/partie : 12000
- Temps moyen/partie : 2.695s
- Différence Elo (approx.) : 56.1

Interprétation. compatible avec 50\% (différence non concluante) RAVE peut être légèrement devant quand l'AMAF local suffit ; l'avantage de GRAVE apparaît surtout quand les AMAF locales sont très bruitées. À budget moyen/élevé, on s'attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 2.695s par partie (ordre de grandeur).

large_scale/uct_vs_grave_100.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 5×5 Budget : 100 itérations/coup Parties : 50
- Score : 20–30 (agent1–agent2) Winrate(agent1) : 40.0% IC95% : [27.6%, 53.8%]
- Simulations moyennes/partie : 2400
- Temps moyen/partie : 0.438s
- Différence Elo (approx.) : -70.4

Interprétation. compatible avec 50\% (différence non concluante) L'algorithme enrichi (AMAF) semble mieux exploiter un budget limité en réutilisant l'information des coups joués dans les simulations. Temps moyen mesuré : 0.438s par partie (ordre de grandeur).

large_scale/uct_vs_grave_200.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 5×5 Budget : 200 itérations/coup Parties : 50
- Score : 39–11 (agent1–agent2) Winrate(agent1) : 78.0% IC95% : [64.8%, 87.2%]
- Simulations moyennes/partie : 4800
- Temps moyen/partie : 0.913s
- Différence Elo (approx.) : 219.9

Interprétation. supérieur à 50\% (gain statistiquement crédible) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À budget moyen/élevé, on s'attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 0.913s par partie (ordre de grandeur).

large_scale/uct_vs_grave_50.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 5×5 Budget : 50 itérations/coup Parties : 50
- Score : 21–29 (agent1–agent2) Winrate(agent1) : 42.0% IC95% : [29.4%, 55.8%]
- Simulations moyennes/partie : 1200
- Temps moyen/partie : 0.217s
- Différence Elo (approx.) : -56.1

Interprétation. compatible avec 50\% (différence non concluante) L'algorithme enrichi (AMAF) semble mieux exploiter un budget limité en réutilisant l'information des coups joués dans les simulations. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.217s par partie (ordre de grandeur).

large_scale/uct_vs_grave_500.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 5×5 Budget : 500 itérations/coup Parties : 50
- Score : 45–5 (agent1–agent2) Winrate(agent1) : 90.0% IC95% : [78.6%, 95.7%]
- Simulations moyennes/partie : 12000
- Temps moyen/partie : 2.245s
- Différence Elo (approx.) : 381.7

Interprétation. supérieur à 50\% (gain statistiquement crédible) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À budget moyen/élevé, on s'attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 2.245s par partie (ordre de grandeur).

large_scale/uct_vs_rave_100.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 5×5 Budget : 100 itérations/coup Parties : 50
- Score : 20–30 (agent1–agent2) Winrate(agent1) : 40.0% IC95% : [27.6%, 53.8%]
- Simulations moyennes/partie : 2400
- Temps moyen/partie : 0.668s
- Différence Elo (approx.) : -70.4

Interprétation. compatible avec 50\% (différence non concluante) L’algorithme enrichi (AMAF) semble mieux exploiter un budget limité en réutilisant l’information des coups joués dans les simulations. Temps moyen mesuré : 0.668s par partie (ordre de grandeur).

large_scale/uct_vs_rave_200.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 5×5 Budget : 200 itérations/coup Parties : 50
- Score : 34–16 (agent1–agent2) Winrate(agent1) : 68.0% IC95% : [54.2%, 79.2%]
- Simulations moyennes/partie : 4800
- Temps moyen/partie : 1.323s
- Différence Elo (approx.) : 130.9

Interprétation. supérieur à 50\% (gain statistiquement crédible) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l’AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À budget moyen/élevé, on s’attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 1.323s par partie (ordre de grandeur).

large_scale/uct_vs_rave_50.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 5×5 Budget : 50 itérations/coup Parties : 50
- Score : 21–29 (agent1–agent2) Winrate(agent1) : 42.0% IC95% : [29.4%, 55.8%]
- Simulations moyennes/partie : 1200
- Temps moyen/partie : 0.327s
- Différence Elo (approx.) : -56.1

Interprétation. compatible avec 50\% (différence non concluante) L’algorithme enrichi (AMAF) semble mieux exploiter un budget limité en réutilisant l’information des coups joués dans les simulations. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d’où l’intérêt d’examiner l’intervalle de confiance. Temps moyen mesuré : 0.327s par partie (ordre de grandeur).

large_scale/uct_vs_rave_500.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 5×5 Budget : 500 itérations/coup Parties : 50
- Score : 41–9 (agent1–agent2) Winrate(agent1) : 82.0% IC95% : [69.2%, 90.2%]
- Simulations moyennes/partie : 12000

- Temps moyen/partie : 2.133s
- Différence Elo (approx.) : 263.4

Interprétation. supérieur à 50\% (gain statistiquement crédible) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À budget moyen/élevé, on s'attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 2.133s par partie (ordre de grandeur).

rave___vs___grave___100.csv

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 5×5 Budget : 100 itérations/coup Parties : 50
- Score : 31–19 (agent1–agent2) Winrate(agent1) : 62.0% IC95% : [48.2%, 74.1%]
- Temps moyen/partie : 0.450s
- Différence Elo (approx.) : 85.0

Interprétation. compatible avec 50\% (différence non concluante) RAVE peut être légèrement devant quand l'AMAF local suffit ; l'avantage de GRAVE apparaît surtout quand les AMAF locales sont très bruitées. Temps moyen mesuré : 0.450s par partie (ordre de grandeur).

rave___vs___grave___200.csv

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 5×5 Budget : 200 itérations/coup Parties : 50
- Score : 27–23 (agent1–agent2) Winrate(agent1) : 54.0% IC95% : [40.4%, 67.0%]
- Temps moyen/partie : 0.919s
- Différence Elo (approx.) : 27.9

Interprétation. compatible avec 50\% (différence non concluante) RAVE peut être légèrement devant quand l'AMAF local suffit ; l'avantage de GRAVE apparaît surtout quand les AMAF locales sont très bruitées. À budget moyen/élevé, on s'attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 0.919s par partie (ordre de grandeur).

rave___vs___grave___50.csv

- Match : **RAVE vs GRAVE** (agent1 = RAVE, agent2 = GRAVE)
- Plateau : 5×5 Budget : 50 itérations/coup Parties : 50
- Score : 24–26 (agent1–agent2) Winrate(agent1) : 48.0% IC95% : [34.8%, 61.5%]
- Temps moyen/partie : 0.232s
- Différence Elo (approx.) : -13.9

Interprétation. compatible avec 50\% (différence non concluante) GRAVE peut stabiliser l'estimation AMAF en s'appuyant sur un ancêtre suffisamment visité, ce qui réduit le bruit des statistiques précoces. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.232s par partie (ordre de grandeur).

test___uct___vs___rave.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 3×3 Budget : 30 itérations/coup Parties : 10
- Score : 8–2 (agent1–agent2) Winrate(agent1) : 80.0% IC95% : [49.0%, 94.3%]
- Simulations moyennes/partie : 240
- Temps moyen/partie : 0.011s
- Différence Elo (approx.) : 240.8

Interprétation. compatible avec 50\% (différence non concluante) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.011s par partie (ordre de grandeur).

uct___vs___grave___100.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 5×5 Budget : 100 itérations/coup Parties : 50
- Score : 29–21 (agent1–agent2) Winrate(agent1) : 58.0% IC95% : [44.2%, 70.6%]
- Temps moyen/partie : 0.430s
- Différence Elo (approx.) : 56.1

Interprétation. compatible avec 50\% (différence non concluante) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. Temps moyen mesuré : 0.430s par partie (ordre de grandeur).

uct___vs___grave___200.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 5×5 Budget : 200 itérations/coup Parties : 50
- Score : 36–14 (agent1–agent2) Winrate(agent1) : 72.0% IC95% : [58.3%, 82.5%]
- Temps moyen/partie : 0.851s
- Différence Elo (approx.) : 164.1

Interprétation. supérieur à 50\% (gain statistiquement crédible) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À budget moyen/élevé, on s'attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 0.851s par partie (ordre de grandeur).

uct___vs___grave___50.csv

- Match : **UCT vs GRAVE** (agent1 = UCT, agent2 = GRAVE)
- Plateau : 5×5 Budget : 50 itérations/coup Parties : 50
- Score : 17–33 (agent1–agent2) Winrate(agent1) : 34.0% IC95% : [22.4%, 47.8%]
- Temps moyen/partie : 0.228s

— Différence Elo (approx.) : -115.2

Interprétation. inférieur à 50\% (perte statistiquement crédible) L'algorithme enrichi (AMAF) semble mieux exploiter un budget limité en réutilisant l'information des coups joués dans les simulations. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.228s par partie (ordre de grandeur).

uct_vs_rave_100.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 5×5 Budget : 100 itérations/coup Parties : 50
- Score : 29–21 (agent1–agent2) Winrate(agent1) : 58.0% IC95% : [44.2%, 70.6%]
- Temps moyen/partie : 0.433s
- Différence Elo (approx.) : 56.1

Interprétation. compatible avec 50\% (différence non concluante) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. Temps moyen mesuré : 0.433s par partie (ordre de grandeur).

uct_vs_rave_200.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 5×5 Budget : 200 itérations/coup Parties : 50
- Score : 34–16 (agent1–agent2) Winrate(agent1) : 68.0% IC95% : [54.2%, 79.2%]
- Temps moyen/partie : 0.801s
- Différence Elo (approx.) : 130.9

Interprétation. supérieur à 50\% (gain statistiquement crédible) Ici, la baseline UCT tient tête voire dépasse la variante AMAF ; cela peut arriver si l'AMAF introduit un biais (corrélations de playout) ou si le budget est suffisamment grand pour que les estimations UCT deviennent fiables. À budget moyen/élevé, on s'attend à une réduction du bruit et à une convergence progressive des méthodes vers des choix similaires (si la politique de rollout est la même). Temps moyen mesuré : 0.801s par partie (ordre de grandeur).

uct_vs_rave_50.csv

- Match : **UCT vs RAVE** (agent1 = UCT, agent2 = RAVE)
- Plateau : 5×5 Budget : 50 itérations/coup Parties : 50
- Score : 17–33 (agent1–agent2) Winrate(agent1) : 34.0% IC95% : [22.4%, 47.8%]
- Temps moyen/partie : 0.223s
- Différence Elo (approx.) : -115.2

Interprétation. inférieur à 50\% (perte statistiquement crédible) L'algorithme enrichi (AMAF) semble mieux exploiter un budget limité en réutilisant l'information des coups joués dans les simulations. À très petit budget, les résultats sont souvent sensibles au hasard des playouts : d'où l'intérêt d'examiner l'intervalle de confiance. Temps moyen mesuré : 0.223s par partie (ordre de grandeur).